

MIDSIZE ENTERPRISE SUMMIT · FALL 2026 · MARCO ISLAND

The Local AI Starter Playbook

Pick the workload. Size the hardware. Ship it safely. Prove it worked.

Companion to “Small Models, Real Work” · Jason Caparoso, CTO & Co-Founder, BlackHawk Data

This is the working version of the session. Ten pages, no theory. Fill it in with a pen, in a meeting, with the people who would actually have to run the thing.

Everything in here runs on open-source software and commodity hardware. Nothing in it requires BlackHawk, or any vendor we partner with.

HOW TO WORK THROUGH IT

1. Score two or three candidate workloads on page 2. Keep the highest scorer; park the rest.
2. Check it against the rent-versus-own table on page 3 before you price any hardware.
3. Use the ninety-day plan on page 6 to prove it on hardware you already own — the first month costs nothing.
4. Only then look at the hardware ladder on page 4 and spend money.
5. Before anything touches production, walk the harness controls and governance wrapper on page 7.
6. If your world includes plant floors, terminals or any enclave with no way out, the edge addendum is on page 8.

1 · The workload filter

A good first workload passes all five tests. Score each candidate one point per test. Five out of five is a build. Three or four is a maybe — fix the gaps first. Two or less, pick something else; you will not be able to defend it later.

TEST	WHAT IT MEANS	POINT
The data is private	Regulated, contracted, or genuinely sensitive. If it already lives in a public SaaS tenant under a signed agreement, local buys you nothing.	
It repeats	Weekly, nightly, forever. One-off work never repays the build.	
It's bounded	One input shape, one output shape. “Anything someone might ask” is a research program, not a first project.	
It ends in an artifact	A file, a memo, a diagram, a ticket, a record. Not a chat answer that evaporates when the tab closes.	
Nobody does it today	The one everyone skips. Work already being done has a baseline you must beat. Work not being done at all does not.	

Candidate scoring sheet

CANDIDATE WORKLOAD	PRIVATE	REPEATS	BOUNDED	ARTIFACT	UNDONE	SCORE

THE SENTENCE THAT GETS IT FUNDED

“This is a capability we do not have today, producing artifacts that do not exist today, on infrastructure that never leaves our trust boundary.”

Do not sell cost avoidance. The token math works, but a good CFO will pick at it and they will be right. Sell the empty folder that is now full.

2 · Rent or own

Local is not cheaper by default. In 2026 it is usually more expensive than a hosted open-weight API. Own the workloads that meet one of these conditions; rent everything else and spend the savings on governance.

OWN IT (RUN IT LOCALLY) WHEN	WHY
The data cannot leave	Regulated data, contractual residency clauses, client prohibitions. This is the strongest reason and usually the only one you need.
There is no egress path at all	Air-gapped and segmented networks — plant floors, terminals, substations, vessels, remote sites. Local is not a preference here; it is the only option.
Latency or availability is a safety property	It has to answer when the WAN is down, because the WAN is what fails during the incident you need it for.
The duty cycle never stops	Nightly sweeps, continuous batch review. Steady, predictable volume is where owning the hardware starts to beat renting it.
You need to own the capability	A task you want to keep when a vendor reprices, sunsets the product, or gets acquired.

RENT IT (HOSTED API) WHEN	WHY
Usage is spiky or occasional	A GPU costs the same idle. An API costs nothing idle. Chat that spikes at 10am and sleeps at 6pm is the worst possible case for owned hardware.
You need the hardest 10%	Deep reasoning, complex coding, long-horizon work. Frontier models are still meaningfully ahead and you are renting that gap by the token.
The data already crossed the boundary	If it lives in a public cloud tenant under an agreement, a second local copy adds governance surface without removing exposure.
You are still finding out whether it works	Prototype against an API. Move it in-house once the workload and the volume are real.

THE HONEST DEFAULT

Most workloads still belong on hosted APIs. Local AI is for the workloads that cannot go there — or that cost too much when they do. If a candidate passes none of the “own it” conditions, stay in the cloud and put your energy into identity, logging and data controls instead.

3 · The hardware ladder

You are not starting at \$100,000. Start at rung zero and climb only when a pilot forces you to. Prices as of September 2026 — GPU and memory street prices are moving monthly and upward; validate before you budget.

RUNG	SPEND	WHAT IT IS	WHAT IT RUNS, AND WHO IT'S FOR
0	\$0	The machine on your desk	16GB of RAM runs a 4B-class model comfortably. One folder, one prompt, one artifact. Every project should start here, and some should end here.
1	Under \$1,500	A spare workstation + 64GB RAM	Mixture-of-experts models in the 20–30B class with expert offload to system memory. Nightly batch jobs nobody has to watch.
2	\$4,000 – \$5,000	128GB unified-memory desktop	Best capacity per dollar. Slower per token than a discrete card, but it holds models the card cannot. Good for private prototyping and edge enclosures.
3	\$6,000 – \$10,000	Single 24–32GB GPU workstation	Real throughput. Agent loops, production sweeps, concurrent jobs. Buy when a pilot has already earned it.
4	\$35,000+	Multi-GPU department server	Only once you have measured concurrency, not guessed at it. A model that fits is not a model that serves forty simultaneous users.

Sizing rules of thumb

- Memory, roughly: FP16 needs about 2GB per billion parameters; 8-bit about 1GB; 4-bit (Q4/GGUF) about 0.5–0.6GB — plus headroom for the context window.
- Mixture-of-experts models change the math: what matters for speed is active parameters, and the idle experts can live in system RAM. Budget 64GB or more of fast system memory if you plan to use offload.
- Size for peak concurrency, not for the demo. Ten people hitting it at 9am is a different machine than one person testing it at 3pm.
- Plan on eighteen months, not thirty-six, and assume near-zero residual value. The resale market that props up datacenter cards does not reach a workstation in a closet.
- Budget 0.25 to 0.5 FTE to run it — patching, model updates, monitoring. This is the line item that quietly kills these projects.

4 · What you are actually building

Four layers, all open source or commodity. The serving layer speaks the same API shape as the cloud providers, so your applications do not care which one answers.

LAYER	WHAT GOES HERE
Application	Chat UI, retrieval over your documents, workflow hooks, the thing that actually produces the artifact.
Retrieval	Embeddings over your own corpus — runbooks, contracts, manuals, tickets. This is where your facts live. Not in the model.
Model	Open-weight, swappable. Small (4-9B) for extraction, classification and routing. Mid (14-32B) for retrieval answers, summarizing and drafting — the mid-market workhorse. Large or MoE for near-frontier quality.
Serving runtime	An OpenAI-compatible endpoint on your own network. Swapping the model behind it is a config change.
Hardware	One workstation, then a department server, then — rarely — a cluster.

Where the “learning” actually comes from

IF YOU NEED...	USE
Facts that change	Retrieval. Point it at your documents. Cheapest, fastest, and the right answer far more often than people expect.
Tone, format, house style	Fine-tuning. 500–1,000 good examples, one GPU, well under an hour. Teaches form, not facts — using it to teach facts is the classic expensive mistake.
Continuity between runs	Memory, governed. Decide deliberately what it is allowed to remember; a memory store that accepts untrusted input is an attack path.

SAY THIS OUT LOUD TO ANYONE WHO ASKS

A model does not get smarter overnight on its own. It is a frozen file. Everything that feels like learning is architecture you built and govern. It learns your environment the way a good new hire does — because you gave it your documentation and your standards.

5 · The first ninety days

Weeks one through four cost nothing and run on hardware you already own. You make the spending decision in month two, with your own evidence instead of a vendor's slide.

Weeks 1-4 — prove it on what you have

- ☐ Install a local runner (LM Studio for a desktop app, Ollama for a command line and an API) on a machine you already own.
- ☐ Pull a 4B-class open-weight model. Ask a colleague or an assistant whether your hardware can handle a bigger one before you fight with it.
- ☐ Make a folder of ten real files from the workload you scored highest on page 2. Real ones, not samples.
- ☐ Ask for one artifact — a memo, a summary, a checklist, a draft. Not a conversation. A file.
- ☐ Run the same job ten times. Write down every place it gets confused.
- ☐ Turn that list into a checklist. The checklist is your eval. You now have the thing most failed pilots never had.
- ☐ Baseline the before-numbers: hours spent, tickets raised, documents reviewed, time to answer. Without a before-number there is no after-story.

Weeks 5-8 — pilot it properly

- ☐ One workload, one team, one machine. Read-only against real systems. Nothing writes anywhere.
- ☐ Retrieval over the documents that matter — start with one document type, not the whole archive.
- ☐ SSO, logging and a written usage policy from day one. “It's in the building” is not an access policy.
- ☐ Every output cites its source. If it cannot cite, it does not publish.
- ☐ Show one artifact to the person who would have to fund this. Something that did not exist a month ago.

Weeks 9-12 — measure and decide

- ☐ Compare honestly against the baseline. Use the eval checklist, not impressions.
- ☐ Run your own seat count, device count and volume through the rent-versus-own table on page 3.
- ☐ Decide one of three things: scale it, keep it hybrid, or shut it down. All three are successful outcomes of a pilot.
- ☐ If a pilot cannot show a measurable delta in ninety days, kill it or re-scope it. Do not let it drift into pilot purgatory.

6 · The harness — five rules

This is the part that separates a useful system from an incident report. None of these are optional, and none of them are expensive.

- ☐ **1. Propose, never execute. The output is a plan, a pull request, a change ticket. A human approves it. Drafting was always the expensive part; approving is fast.**
- ☐ **2. Read-only by default. The collection identity gets read commands and nothing else. Write access is a separate credential, separately scoped, separately logged, used only on the far side of a human approval.**
- ☐ **3. Cite, or do not ship. Every claim traces to a source, a command and a timestamp. Hallucinated documentation is worse than none, because a wrong document that looks authoritative gets trusted.**
- ☐ **4. Cap the report at five findings, ranked, each with a proposed action. A forty-item report becomes the new thing nobody reads.**
- ☐ **5. Treat every ingested artifact as untrusted input. Log lines, banners, ticket bodies, document text — all reachable by people who do not work for you, all being fed to a system that holds credentials. Put an egress allowlist on the harness itself; the model is local but its tooling pulls packages from the internet.**

THE THING PEOPLE GET WRONG

Running the model locally reduces disclosure risk. It does nothing at all for action risk. It can even make teams less careful, because the local deployment feels like a security control when it is not one. The controls that matter live in the harness, not in where the GPU sits.

Governance wrapper — do not skip

- ☐ Identity: every request tied to a person through your existing SSO.
- ☐ Logging: every prompt and every response retained, searchable, with a defined retention period.
- ☐ Data controls: what corpora this instance may retrieve from, written down and enforced.
- ☐ An owner: a named human responsible for the model version, the patch cadence, and the eval results.
- ☐ A decommission path: how this gets turned off and what happens to the logs and the corpus if it does.

7 · Edge and air-gapped addendum

For terminals, plant floors, substations, transit systems, remote sites and any enclave with no egress path. Everything on page 7 still applies — and applies harder. These are the additional controls.

Placement and boundaries

- ☐ Place the inference host where your architecture says a general-purpose server belongs — for most designs that is the industrial DMZ or an equivalent enclave, not the control network.
- ☐ The host is an asset like any other: named owner, patch cadence, hardened baseline, monitored, inventoried.
- ☐ No new egress path is created for the model. If the design requires one, the design has changed and needs a fresh review.
- ☐ Read-only, propose-only, with no credential that can act on equipment. Next to a controller this is a safety requirement, not a preference.
- ☐ Transfers in and out follow the existing controlled-media or one-way-transfer process. The model does not get an exception.

Model lifecycle in an air gap

- ☐ Treat model weights like firmware: staged, validated against your own eval set, versioned, and moved under change control.
- ☐ Stamp every artifact the model produces with the model version and an as-of date. Whoever reads it in eight months needs to know what it knew.
- ☐ Plan the update path before deployment, not after. Expect to refresh two or three times a year, not continuously.
- ☐ Keep a rollback copy of the previous model version and its eval results.
- ☐ Re-run your eval after every model change. A quiet quality regression in an enclave nobody can reach from the outside is very hard to notice.

Good first workloads at the edge

- Documents before alarms. Retrieval over SOPs, vendor manuals, as-builts and drawings — so a shift finds the right page instead of the right binder. Touches nothing operational and usually clears change control in one meeting.
- Shift handover drafted from the log, for a human to correct and sign.
- Alarm triage and plain-language explanation as a suggestion alongside the alarm — never as an action.
- Pre-filled incident and maintenance records, submitted as drafts.

8 · Five ways this goes wrong

Say these before your CFO does. Volunteering your own failure modes is what makes the rest of your numbers believable.

FAILURE	THE FIX
Counting the hardware, not the humans	Budget 0.25–0.5 FTE explicitly. It is the omitted line that kills these projects.
No eval	Write the test before you build the thing. Around 88% of agent pilots never reach production, and the most-named blocker is that nobody defined success up front.
Dumping everything into context	Quality falls off as the window fills and hallucination rates climb. Retrieve what matters; do not shovel.
Confusing speed with capability	Techniques that make a model faster produce identical output. They do not turn a small model into a large one.
Assuming portability	Swapping models is two lines of config. Your fine-tune does not transfer and your prompts need re-tuning. What you keep — retrieval, tools, evals, integration — is 80–90% of the value anyway.

9 · What to measure

Fill the left column in before you launch. You cannot reconstruct a baseline afterwards.

METRIC	BASELINE (BEFORE)	AFTER 90 DAYS
Hours spent on the task per week		
Volume handled per week		
Time to first answer		
Tickets deflected / reviews avoided		
Rework or correction rate		
Artifacts produced that did not exist before		
Run cost (hardware amortized + power + FTE)		

10 · Plain-language glossary

For the conversation with the person who signs the purchase order.

TERM	WHAT IT MEANS
Open-weight model	A model whose file you can download and run yourself. Check the licence before commercial use; most of the useful ones are permissive.
Parameters (7B, 32B...)	Roughly, the size of the model. More capacity for harder tasks, more memory required.
Mixture of experts (MoE)	A model that only activates a fraction of itself for each word. Carries the knowledge of a very large model with the compute cost of a small one — the single biggest reason local became practical.
Quantization (Q4, Q8, GGUF)	Compression. Q4 is easier to run, Q8 keeps more quality. It is what lets a large model fit a normal machine.
Context window	How much material the model can consider at once. Bigger is not automatically better — quality degrades as it fills.
Retrieval / RAG	Searching your own documents by meaning and handing the relevant pieces to the model. This is where your facts should live.
Embeddings	The numeric representation that makes search-by-meaning work. A separate, very small model usually does this.
Fine-tuning / LoRA	Teaching a model your tone, format and house style with a few hundred examples. Not a way to teach it facts.
Eval	A small repeatable test that tells you whether the model did the job well enough. The most skipped and most important artifact in the whole project.
Serving runtime	The software that makes the model answerable over an API on your network — the part your applications talk to.
Tokens	The chunks of text a model reads and writes. Locally you care about speed and memory; on an API you are billed by them.
Air gap	A network with no path to the internet by design. A model can be carried in on controlled media; nothing calls out.

BRING US YOUR NUMBERS

Free AI workload and sizing review. Bring one candidate workload from page 2 and your seat or device count, and we will walk it through the filter and the cost model with you — including when the honest answer is that you should rent, or that your existing monitoring already does it.

BlackHawk Data · Booth [##], Solutions Pavilion · blackhawk11.com · [email]

We partner with Cloudflare, Cisco and Fortinet. Nothing here requires any of them.